



UNIVERSIDADE FEDERAL DE SERGIPE
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

**RECUPERAÇÃO EFICIENTE DE CONTEXTO EM ARTIGOS
CIENTÍFICOS SOBRE MODELOS DE LINGUAGEM,
RECUPERAÇÃO DE INFORMAÇÃO E RETRIEVAL-AUGMENTED
GENERATION (RAG).**

CLARIANE MARIA OLIVEIRA LISBOA
IRIS AZENATE FERREIRA
JOSÉ NILTON DOS SANTOS OLIVEIRA
LEONARDO SOUZA SILVA
THIAGO DE JESUS DOS SANTOS
VICTÓRIA CARDOSO DOS SANTOS

INTRODUÇÃO



Clariane Maria Oliveira Lisboa

- O trabalho aborda a recuperação de contexto em artigos científicos relacionados a Modelos de Linguagem de Grande Escala (LLMs), Recuperação de Informação e Retrieval-Augmented Generation (RAG).
- Foi utilizado um corpus de artigos da ACL Anthology, considerando títulos e resumos.

- O experimento compara três configurações:

1. Busca Linear;
2. Índice Invertido;
3. Merge Sort + Busca.

- A comparação considera:

1. Tempo de Consulta;
2. Número de Comparações;
3. Precision@5.

Corpus e Preparação



Íris Azenate Ferreira

```
○ QUERY_TERMS = [
    "retrieval augmented generation",
    "information retrieval",
    "large language model",
]

N_ARTIGOS_COLETA = 18000

def score_relevancia(texto, termos):
    return sum(texto.count(termo) for termo in termos)

def coletar_corpus_acl(termos, n=100):
    anthology = Anthology.from_repo()
    candidatos = []

    for paper in anthology.papers():
        if paper.abstract is None or paper.title is None:
            continue

        titulo = str(paper.title)
        abstract = str(paper.abstract)
        texto = (titulo + " " +
            abstract).lower()

        score = score_relevancia(texto,
            termos)

        if score > 0:
            candidatos.append({
                "id": paper.full_id,
                "title": titulo,
                "abstract": abstract,
                "year": paper.year,
                "doi": paper.doi,
                "url": paper.web_url,
                "_score": score
            })

        candidatos.sort(
            key=lambda x: (x["_score"], x["year"] or 0),
            reverse=True
        )

        top = candidatos[:n]

        df = pd.DataFrame(top).drop(columns=
            ["_score"])
        return df

    df = coletar_corpus_acl(
        QUERY_TERMS,
        N_ARTIGOS_COLETA
    )

    print("Artigos coletados:", len(df))
    display(df[["title", "year", "url"]])
```

Normalização e Chunking



Íris Azenate Ferreira

```
def normalizar(texto):  
    texto = str(texto).lower()  
    texto = re.sub(r"^[^a-z0-9\s]", " ", texto)  
    texto = re.sub(r"\s+", " ", texto)  
    return texto.strip()
```

```
from nltk.stem import PorterStemmer  
stemmer = PorterStemmer()
```

```
def tokenizar(texto):  
    tokens = normalizar(texto).split()  
    return [stemmer.stem(t) for t in tokens]
```

```
# Chunking  
def preparar_documentos(df):  
    documentos = []
```

```
for _, row in df.iterrows():  
    texto = (  
        str(row["title"]) + " " +  
        str(row["abstract"])  
    )  
  
    documentos.append({  
        "id": row["id"],  
        "title": row["title"],  
        "text": normalizar(texto),  
        "tokens": tokenizar(normalizar(texto)),  
        "url": row["url"],  
        "year": row["year"]  
    })  
    return documentos
```

```
documentos = preparar_documentos(df)
```

```
print("Documentos  
preparados:", len(documentos))  
print("\nExemplo:")  
print(documentos[0]["title"])  
print(documentos[0]["text"]  
      [:500], "...")
```

```
print("Estratégia de chunking:")  
print("- 1 artigo = 1  
      documento/chunk")  
print("- Representação = título +  
      abstract")  
print("- N pequeno = 500 artigos")  
print("- N grande = 1000 artigos")
```



Clariane Maria Oliveira Lisboa

DEFINIÇÃO FORMAL

- A recuperação recebe um corpus de documentos, uma consulta e o número de resultados desejados. Para cada documento é calculada uma pontuação (score) em relação à consulta. Ao final, são retornados os k documentos com maior pontuação.

- Entrada

(C, q, k)

- $C \rightarrow$ corpus de documentos
- $q \rightarrow$ consulta
- $k \rightarrow$ número de documentos a retornar

- Saída

$\text{Topk}(C, q)$

- $\rightarrow$ documentos com maior score, em ordem decrescente.

Objetivo: recuperar e ordenar os documentos do corpus de acordo com sua pontuação em relação à consulta.



Clariane Maria Oliveira Lisboa

PONTUAÇÃO LEXICAL (SCORE)

- A pontuação de um documento foi calculada pela frequência dos termos da consulta presentes no texto do documento.
- Score = soma das ocorrências dos termos da consulta no documento

Exemplo:

Consulta: retrieval language

Documento: retrieval retrieval methods for language models

- retrieval → 2 ocorrências
- language → 1 ocorrência
- Score = 2 + 1 = 3

Quanto maior o score, maior a prioridade do documento na ordenação dos resultados.



Clariane Maria Oliveira Lisboa

CONSULTAS UTILIZADAS

Foram definidas 10 consultas relacionadas aos temas do corpus:

QUERIES = {

"Q1": "What information retrieval methods are used in RAG systems?",

"Q2": "Which studies compare sparse retrieval and dense retrieval?",

"Q3": "What semantic search techniques are used to retrieve context for language models?",

"Q4": "What reranking methods are used in retrieval systems for LLMs?",

"Q5": "What metrics are used to evaluate Retrieval-Augmented Generation systems?",

"Q6": "Which papers use embeddings for document retrieval?",

"Q7": "What techniques are used to select the most relevant documents or excerpts for a language model?",

"Q8": "Which methods were compared for context retrieval?",

"Q9": "What limitations of RAG systems are reported in the articles?",

"Q10": "What chunking strategies are used in document retrieval?"

}

BUSCA LINEAR



Clariane Maria Oliveira Lisboa

- A Busca Linear percorreu todos os documentos do corpus para cada consulta.

- **Funcionamento:**

1. **Recebe o corpus (C), a consulta (q) e (k);**
2. **Percorre cada documento do corpus;**
3. **Calcula o score do documento;**
4. **Armazena documento e score;**
5. **Ordena os documentos por score decrescente;**
6. **Retorna os (k) primeiros documentos.**

- **Pseudocódigo**

Para cada documento d em C:

score $\leftarrow$ score_documento(d, q)

armazenar (score, d)

ordenar por score decrescente

retornar os k primeiros

Característica principal:

→ Todos os documentos foram avaliados antes da seleção dos resultados.

MERGE SORT



Leonardo Souza Silva

função merge_sort(lista)

se tamanho(lista) ≤ 1

retorne lista

meio = tamanho(lista) / 2

esquerda = merge_sort(lista[0 até meio])

direita = merge_sort(lista[meio até o fim])

retorne merge(esquerda, direita)

função merge(esquerda, direita)

resultado = lista vazia

enquanto esquerda e direita tiverem elementos:

se esquerda[0] $\geq$ direita[0]:

adicione esquerda[0] ao resultado

remova o elemento da esquerda

senão:

adicione direita[0] ao resultado

remova o elemento da direita

adicione o restante da esquerda e da direita ao resultado

retorne resultado

CORRETUDE



Leonardo Souza Silva

Indução forte - Recursão

função merge_sort(lista)

se tamanho(lista) ≤ 1

retorne lista

meio = tamanho(lista) / 2

esquerda = merge_sort(lista[0 até meio])

direita = merge_sort(lista[meio até o fim])

retorne merge(esquerda, direita)

Invariante de laço - while

função merge(esquerda, direita)

resultado = lista vazia

enquanto esquerda e direita tiverem elementos:

se esquerda[0] $\geq$ direita[0]:

adicione esquerda[0] ao resultado

remova o elemento da esquerda

senão:

adicione direita[0] ao resultado

remova o elemento da direita

adicione o restante da esquerda e da direita ao resultado

retorne resultado

INDUÇÃO FORTE



Leonardo Souza Silva

Passo 1 - Hipótese de indução

- ✓ Verdade para um k , onde $k < N$, onde N é o tamanho da lista.

Passo 2 - Prova o caso base

Se a lista contém zero ou um elemento ($N = 0$ ou $N = 1$)

*“função `merge_sort(lista)`
se `tamanho(lista) <= 1`
retorne `lista`”*

Passo 3 - Passo indutivo

Considere uma lista de entrada N com cardinalidade arbitrária $N > 1$.

Partição Esquerda: de tamanho aproximado $N/2$.

Partição Direita: de tamanho aproximado $N/2$.

Como $1 \leq PE < N$ e $1 \leq PD < N$

Temos que PE e PD são maiores ou iguais que o caso base, e menores que N , o que prova a hipótese de indução

- ✓ Verdade para um k , onde $k < N$

INVARIANTE DE LAÇO



Leonardo Souza Silva

INICIALIZAÇÃO

✓ As listas já estão ordenadas da parte 1

MANUTENÇÃO

A cada passo, comparamos as frentes esquerda[0] e direita[0]

1. Vetores ordenados decrescentemente possuem maiores valores no início
2. Adicionar esses elementos ao fim da lista só cresce o vetor resultado e não altera a ordenação

TÉRMINO

Processo de manutenção ocorre até as listas ficarem vazias, não tendo mais elementos o while acaba.

Elementos restantes são adicionados a lista, como já estão ordenados, vetor resultado só aumenta mantendo ordenação.



José Nilton dos Santos Oliveira

Índice Invertido - Introdução

D0 retrieval augmented
Generation language

D1 dense retrieval and
language representation

D2 large language model
pretraining

`indice(termo) → postings`

retrieval [0, 1]

augmented [0]

generation [0]

language [0, 1, 2]

model [0, 2]

dense [1]

representation [1]

large [2]

pretraining [2]

"language" está nos 3 docs → lista longa. "dense" é exclusivo de D1 → lista curta.



José Nilton dos Santos Oliveira

Termo entra uma vez por documento

```
palavras = set(tokenizar(doc["text"]))
```

Sem set()

- "language" repetido 10x no texto
- → índice do documento é inserido 10x na mesma lista de postings
- → infla o custo de construção e de busca sem agregar informação

Com set()

- Termos únicos por documento → cada doc entra no máximo 1x por posting
- Índice responde "contém ou não?" — não é contagem
- Frequência real fica nos tokens do doc, usada depois só nos candidatos



José Nilton dos Santos Oliveira

Índice Invertido - Construção e consulta

`construir_indice_invertido`

```
indice = {}  
para cada documento i:  
    para cada termo único  
    do documento i:  
        adicione i a  
        indice[termo]  
  
retorne indice
```

Uma varredura completa do corpus — feita 1 vez.



`busca_indexada`

```
candidatos = U indice[termo]  
    para termo da consulta  
  
para i em candidatos:  
    score(i, consulta)  
  
ordene por score  
retorne top-k
```

Só avalia os candidatos — nunca o corpus inteiro.

ANÁLISE ASSINTÓTICA



Victória Cardoso dos Santos

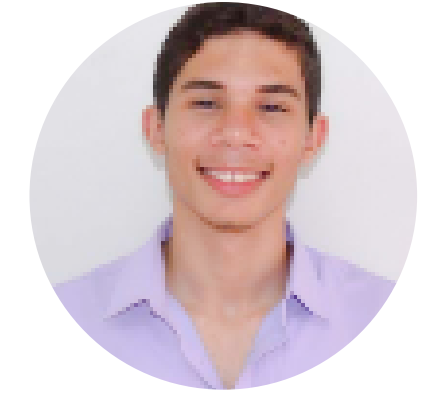
Quadro 1 – Síntese da análise assintótica

Algoritmo implementado	Melhor caso	Caso médio/pior	Espaço
Busca Linear	$\Theta(mL + N)$	$\Theta(mL + N \log N)$	$O(N)$
Índice Invertido	$O(m)$	$O(m+P_q+mL_r+r \log r)$ aproximando $\Theta(mL + N \log N)$ no pior caso ($r = N$)	$O(r)$ durante a consulta + $O(L)$ para armazenar o índice
Merge Sort	$\Theta(N \log N)$	$\Theta(N \log N)$	$O(N)$ auxiliar

- Nota: Busca Linear e Índice Invertido já incluem a ordenação em sua própria função, por isso aparecem como custo total. O Merge Sort é uma função de ordenação separada, na Configuração 3, a pontuação $\Theta(mL)$ precede sua chamada, resultando em custo total $\Theta(mL + N \log N)$.

Legenda: N = documentos; L = tokens; m = termos da consulta; r = documentos candidatos; L_r = tokens dos candidatos; P_q = itens percorridos nas listas do índice.

DESENHO EXPERIMENTAL



Thiago de Jesus dos Santos

Simulação de Monte Carlo

50 Replicações

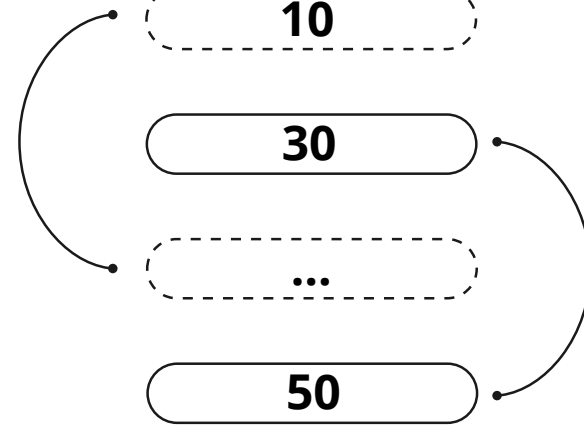
1

10

30

...

50



Amostras

500

1000

Métodos de busca

Busca Linear

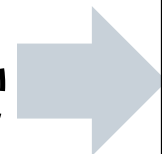
Índice Invertido

Merge Sort

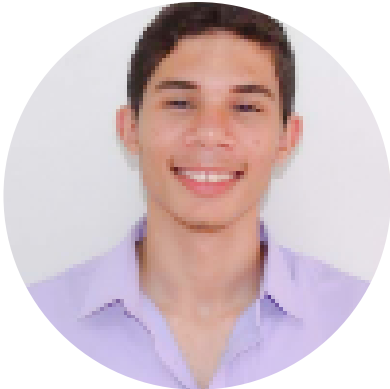
Análise de Métricas

- **Precision@5**
- **Tempo de Execução**
- **Pico de memória**

Dados



RESULTADOS DO EXPERIMENTO



Thiago de Jesus dos Santos

TABELA 1 - ANÁLISE DE PRECISÃO E CUSTO COMPUTACIONAL DOS MÉTODOS DE BUSCA

Corpus	Método	Tempo pré-proc. (s)	Tempo consulta (s)	Tempo total (s)	Comparações	Precision @5	Taxa de resultado vazio	Taxa sem item relevante	Memória média (MB)	Pico memória (MB)
500	Busca Linear	1,5582	0,6271	2,1853	500,0	0,46	0,00%	10,00%	0,0110	0,0123
500	Merge Sort	1,5582	0,6271	2,1853	500,0	0,46	0,00%	10,00%	0,0126	0,0135
500	Índice Invertido	1,5582	0,5127	2,0709	403,5	0,46	0,00%	10,00%	0,0388	0,0438
1000	Busca Linear	3,1848	1,2721	4,4567	1000,0	0,40	0,00%	10,00%	0,0221	0,0243
1000	Merge Sort	3,1848	1,2683	4,4529	1000,0	0,40	0,00%	10,00%	0,0257	0,0268
1000	Índice Invertido	3,1848	1,0207	4,2053	779,4	0,40	0,00%	10,00%	0,0488	0,0557

Os resultados indicam tendências, mas não podem ser generalizados diretamente para outros contextos.



Victória Cardoso dos Santos

ESCALABILIDADE E TRADE-OFF

Quadro 2 – Síntese trade-off.

Algoritmo implementado	Vantagem	Custo ou desvantagem
Busca Linear	Não exige estrutura prévia	Percorre todo o conjunto a cada consulta
Índice Invertido	Restringe a busca aos candidatos	Exige construção e espaço $O(L)$
Merge Sort	Garante $\Theta(N \log N)$ em todos os casos	Não melhora a ordem do sort() no caso médio/pior

- A Busca Linear é uma alternativa simples para conjuntos menores de documentos;
- O custo de construção do índice compensa principalmente quando ele é reutilizado em várias consultas;
- Merge Sort garante uma complexidade de ordenação previsível em todos os casos.

RELAÇÃO COM RAG



Victória Cardoso dos Santos

Quadro 3 – Síntese relação com o RAG..

Aspecto	Impacto
Top-k recuperado	Forma o contexto inserido no prompt
Aumento de k	Maior cobertura, mas mais tokens
Chunks maiores	Mais informação, mas podem incluir conteúdo irrelevante
Ordem dos documentos	Pode influenciar o uso do contexto pelo modelo
Busca lexical	Pode não identificar relações semânticas

- Trade-off no RAG: qualidade e cobertura × tempo, memória e custo de geração..

AMEAÇAS À VALIDADE



Victória Cardoso dos Santos

Quadro 3 – Síntese relação com o RAG..

Tipo	Ameaça identificada
Top-k recuperado	Apenas 10 consultas, apesar das 30 repetições
Aumento de k	Busca lexical avaliada por gabarito semântico automático
Chunks maiores	Títulos e resumos em inglês da ACL Anthology
Ordem dos documentos	Somente dois tamanhos de conjunto
Busca lexical	Hardware, Python, bibliotecas e processos em execução
Apoio de IA generativa	O código sugerido pode parecer correto, mas conter erros ou não cobrir todos os casos

- Trade-off no RAG: qualidade e cobertura × tempo, memória e custo de geração..